

Linear regression

Instructor: Roberto Colomboni

version of 18 September 2026

In the first lecture, we introduced the mathematical tools for describing predictions and differentiating them with respect to adjustable parameters. We now use those tools to fit a prediction rule to data. The same sequence will recur throughout the course: choose a family of functions, specify a loss, and adjust the parameters to reduce the corresponding empirical loss.

In the sequel, input feature vectors are rows, weights for a single output are columns, and argmin denotes a set of minimisers. The notation $\mathbb{R}^d$ states the number of coordinates; $\mathbb{R}^{1 \times d}$ and $\mathbb{R}^{d \times 1}$ make the declared orientation explicit when needed.

1 Regression: modelling a relationship and fitting a predictor

Suppose that we have $N \geq 1$ observations,

$$(x_1, y_1), \dots, (x_N, y_N),$$

where the index i identifies an observation, x_i is its input and y_i is its observed outcome, also called the *target*. For now, both are real numbers. *Regression* aims to predict a numerical outcome from an input.

A useful starting point is to model each outcome as a systematic relationship plus noise:

$$y_i = f_{\text{true}}(x_i) + \varepsilon_i, \quad i = 1, \dots, N. \quad (1)$$

In (1), f_{true} is an unknown function describing the relationship, and ε_i represents variation around it. Such variation may arise from measurement error or factors not recorded in the input. We observe the pairs $(x_1, y_1), \dots, (x_N, y_N)$, but not the function f_{true} or the individual noise terms.

We might expect the relationship to be approximately described by a simple function. For example, an *affine* function of a scalar input has the form

$$f(x) = ax + b, \quad a, b \in \mathbb{R}.$$

Its graph is a straight line: a is the slope and b the intercept. If the underlying relationship f_{true} is affine, the model becomes

$$y_i = ax_i + b + \varepsilon_i, \quad i = 1, \dots, N. \quad (2)$$

Under (2), our task is to estimate a, b from the observed pairs

$$(x_1, y_1), \dots, (x_N, y_N).$$

We refer to this collection of observed input–target pairs as the *observed data*, or the *dataset*. If we knew the noise terms, we could subtract them from the targets and obtain points on the unknown line. Since we do not know them, we will compare candidate lines by how closely their values match the observed targets.

A *predictor*, also called a *prediction rule*, is a function $f : \mathbb{R} \rightarrow \mathbb{R}$ that assigns a prediction to each input. At the observed input x_i , the prediction is $f(x_i)$. With affine predictors, fitting means choosing the slope and intercept of a candidate line using the observed data. Under the assumed model, the fitted line is our estimate of f_{true} .

More generally, even if we do not know whether f_{true} is affine, we can still choose the family of affine functions and fit one to the observed data: the fitted function then serves as an *approximation* to the unknown relationship.

A small example. Consider these five observations:

i	1	2	3	4	5
x_i	-2	-1	0	1	2
y_i	-1.5	0.5	1	2.5	7.5

We use this dataset throughout the scalar-input worked examples.

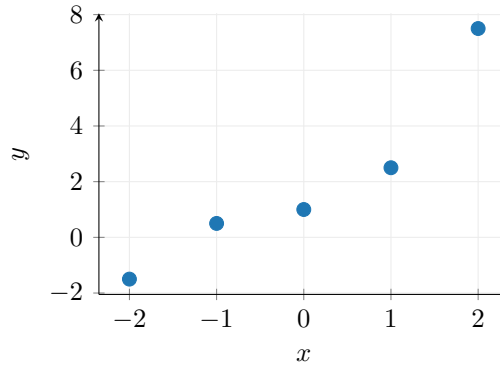


Figure 1: Five observations: each point records an input x_i and its target y_i . We seek a function whose predictions are close to these targets.

The five points are not collinear, so no straight line can pass through all of them. We can nevertheless compare different lines as approximations. As a first candidate, consider

$$f(x) = 1 + 2x. \quad (3)$$

The predictor in (3) has intercept 1 and slope 2. Its predictions at the five inputs are

$$-3, \quad -1, \quad 1, \quad 3, \quad 5.$$

Only the third prediction matches its target exactly. The other predictions lie above or below their targets, as shown in Figure 2.

How should we compare this candidate with another line? We need to specify how prediction errors are penalised and combine the penalties into a numerical criterion. A *loss function* will make this choice precise and allow us to fit the parameters systematically.

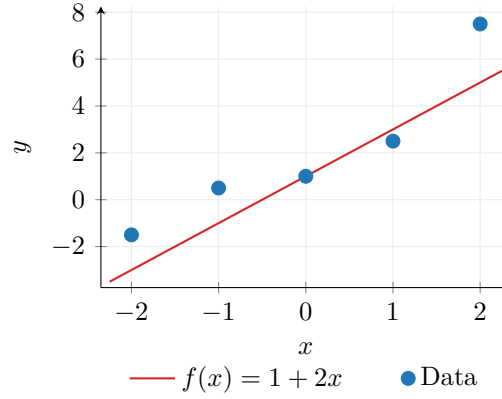


Figure 2: A first guess, $f(x) = 1 + 2x$, plotted against the five data points. Choosing a line gives a prediction at every input, including inputs not present in the dataset.

2 Quantifying prediction errors with a loss

Compare our first guess $f(x) = 1 + 2x$ with $g(x) = 1 - x$. The vertical segments in Figure 3 join each target to its prediction. Their lengths show the prediction errors.

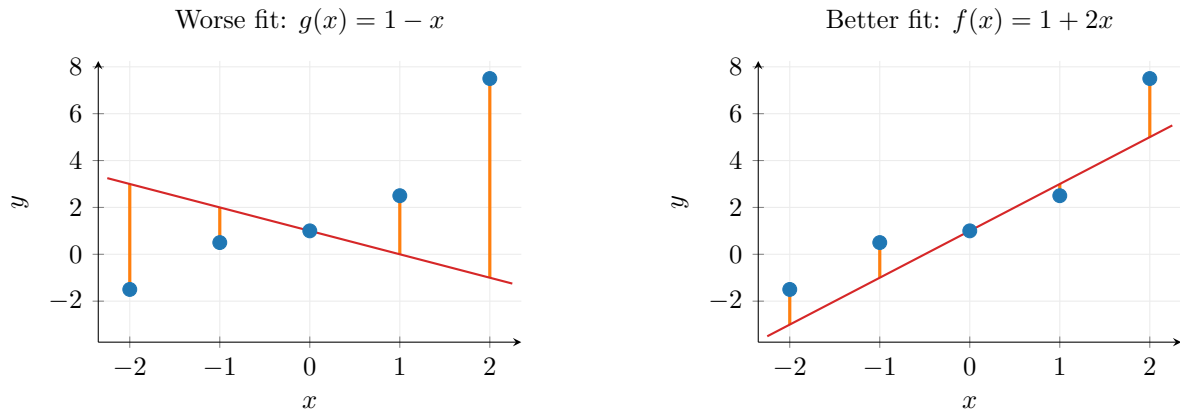


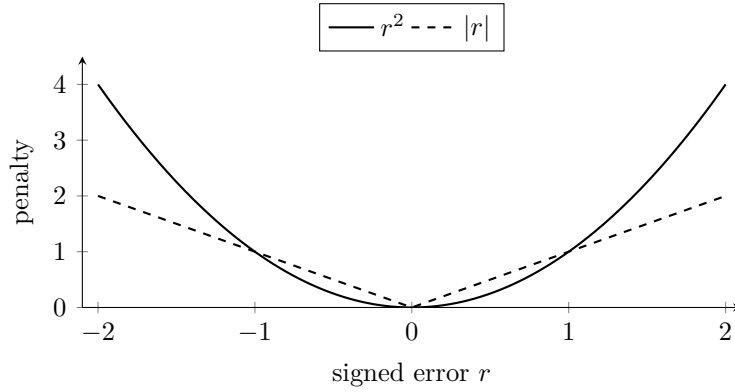
Figure 3: The orange segments show the vertical differences between predictions and targets. Squaring their lengths and adding gives loss 101 for g and loss 11 for f .

A *loss function* assigns a numerical penalty to a prediction and a target. We put the prediction first and the target second. Two choices are the *absolute loss* and the *squared loss*:

$$\ell_{\text{abs}}(\hat{y}, y) := |\hat{y} - y|, \quad \ell_{\text{sq}}(\hat{y}, y) := (\hat{y} - y)^2. \quad (4)$$

Both losses in (4) are nonnegative and equal zero exactly when the prediction equals the target. They differ in how they penalise errors. Doubling the magnitude of an error doubles its absolute loss but quadruples its squared loss.

Writing $r = \hat{y} - y$, the two penalties are $|r|$ and r^2 . The first has a corner at zero, while the second is differentiable everywhere, with derivative $2r$ with respect to r . The corner does not make absolute error an invalid objective, but it means that ordinary derivatives alone will not necessarily settle every optimisation question. We return to this in Exercises 4 and 5.



2.1 From individual errors to an objective on parameters

Let the column vector $\boldsymbol{\theta} \in \mathbb{R}^p$ collect the p adjustable parameters of a family of predictors. We write $f_{\boldsymbol{\theta}}$ for the function associated with the parameter choice $\boldsymbol{\theta}$. For fixed observations, the *sum-of-squared-errors objective* is

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) := \sum_{i=1}^N \ell_{\text{sq}}(f_{\boldsymbol{\theta}}(x_i), y_i) = \sum_{i=1}^N (f_{\boldsymbol{\theta}}(x_i) - y_i)^2. \quad (5)$$

The function $\mathcal{L}_{\text{sq}}$ is the *empirical loss function*, briefly discussed in Lecture 1, associated with the predictor family $f_{\boldsymbol{\theta}}$, the loss function ℓ_{sq} , and the dataset $(x_1, y_1), \dots, (x_N, y_N)$. It aggregates the losses over the observed dataset and is the objective that we minimise.

The loss ℓ_{sq} takes one prediction and one target. The empirical loss $\mathcal{L}_{\text{sq}}$ takes the parameters, uses them to make all the predictions, and adds the resulting losses.

The subscript serves as a reminder of the choice of loss. When we need to distinguish several predictor families, we add a superscript identifying the family to the objective, writing, for example, $\mathcal{L}_{\text{sq}}^{\text{const}}$, $\mathcal{L}_{\text{sq}}^{\text{slope}}$, or $\mathcal{L}_{\text{sq}}^{\text{aff}}$. When the family is clear from the context, we omit the superscript. Choosing the predictor family and choosing the loss are separate decisions.

Throughout an optimisation calculation, the inputs and targets are fixed, while the parameters vary.

We call

$$r_i(\boldsymbol{\theta}) := f_{\boldsymbol{\theta}}(x_i) - y_i \quad (6)$$

the *residual*, using the sign convention “prediction minus target”. Then $\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \sum_{i=1}^N r_i(\boldsymbol{\theta})^2$. Positive residuals in (6) indicate overprediction, negative residuals underprediction. A residual is computable once we choose the predictor.

When evaluating our performance, we do not want opposite signed errors to simply cancel: in a regression problem, we consider both overprediction and underprediction as mistakes, and this is one reason for applying a nonnegative penalty (such as the square of the absolute value) before summing.

Worked comparison. For the two predictors compared in Figure 3, the residuals from (6) are

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	sum of squares
$f(x_i) - y_i$	-1.5	-1.5	0	0.5	-2.5	11
$g(x_i) - y_i$	4.5	1.5	0	-2.5	-8.5	101

Squared error agrees with our visual comparison: f fits these observations better than g .

2.2 The empirical loss: other conventions and interpretation

In the literature, other common objectives, corresponding to different normalisations, are

$$\frac{1}{N}\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}), \quad \frac{1}{2}\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}), \quad \frac{1}{2N}\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}).$$

For a fixed dataset, all have exactly the same minimisers. Indeed, multiplying an objective by a positive constant preserves every comparison between its values. Their *gradients*, however, are multiplied by that constant too.

We remark that *fitting the observed data* and *predicting accurately on unseen observations* are different questions. In particular, a small value of (5) is a statement about the observations in that sum, i.e., the already seen data, and is not by itself a guarantee of accurate predictions on unseen data.

The ability of a fitted predictor to perform well on unseen data is called *generalisation*. Later in the course, we will see how *underfitting* and *overfitting* can prevent good generalisation. Our task in this lecture is limited to understanding the fitting calculation.

3 Choosing a family of prediction functions

The goal of this section is to use derivatives/gradients to compute the best predictor in the chosen class that fits the observed data according to the chosen loss. We have already considered straight-line predictors, but to learn the ropes we compare that family with two simpler families, each imposing a restriction on the coefficients:

family	prediction at x	parameters
constant	b	$b \in \mathbb{R}$
line through the origin	ax	$a \in \mathbb{R}$
general straight line	$b + ax$	$(b, a) \in \mathbb{R}^2$

The slope a controls the change in the prediction when x increases by one. The intercept b is the prediction at $x = 0$. The term *bias* is also used for an intercept. Technically, a straight line with a nonzero intercept is an *affine* function of x , rather than a linear map in the strict linear-algebra sense.

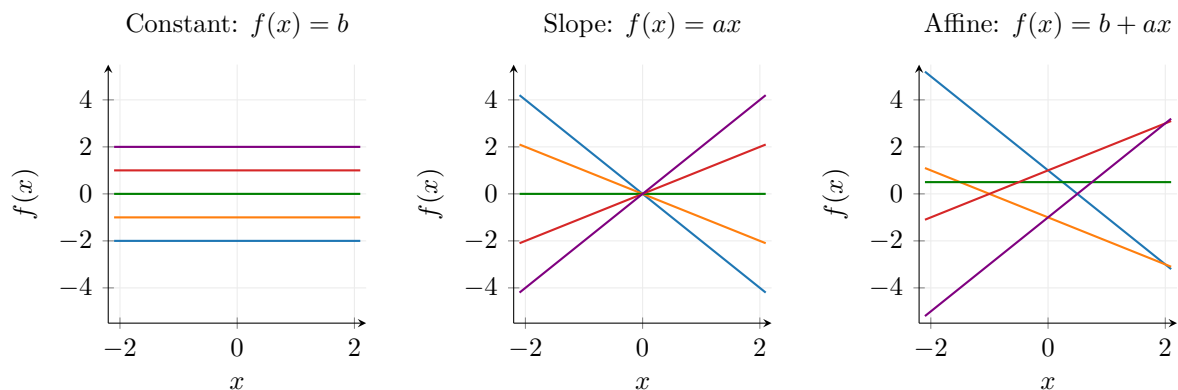


Figure 4: Different parameter choices give different functions within each family. Changing b moves a line vertically; changing a changes its slope. The affine family allows both to vary.

We remark that choosing the family of predictors and choosing its parameters are separate decisions: a constant predictor cannot change with x , no matter how well we choose b ; a line

through the origin is forced to predict zero at $x = 0$; a general straight line removes those two restrictions, but still cannot bend.

Each of the families above contains infinitely many functions. We seek parameter values that minimise the empirical loss. As in Lecture 1, we write

$$\boldsymbol{\theta}^* \in \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^p} \mathcal{L}_{\text{sq}}(\boldsymbol{\theta})$$

for a minimiser.

We recall that, by Fermat's theorem, the gradient of a differentiable objective is zero at a local minimum (i.e., the point is stationary), and that this condition is necessary but not sufficient: a stationary point need not be a minimum. So, our strategy will be to first set the gradient to zero to find the stationary points, and then understand whether or not the points we have found this way are in fact the minimizers we were seeking.

3.1 The best constant predictor

The constant predictor $f_b^{\text{const}} : \mathbb{R} \rightarrow \mathbb{R}$ ignores its input:

$$f_b^{\text{const}}(x) := b. \quad (7)$$

Using (5), its squared-error objective is the one-variable function

$$\mathcal{L}_{\text{sq}}^{\text{const}}(b) := \sum_{i=1}^N \ell_{\text{sq}}(f_b^{\text{const}}(x_i), y_i) = \sum_{i=1}^N (b - y_i)^2. \quad (8)$$

Figure 5 connects a horizontal prediction line to a point on the graph of the objective (8). Changing b moves the prediction line vertically; on the right, b is the horizontal coordinate.

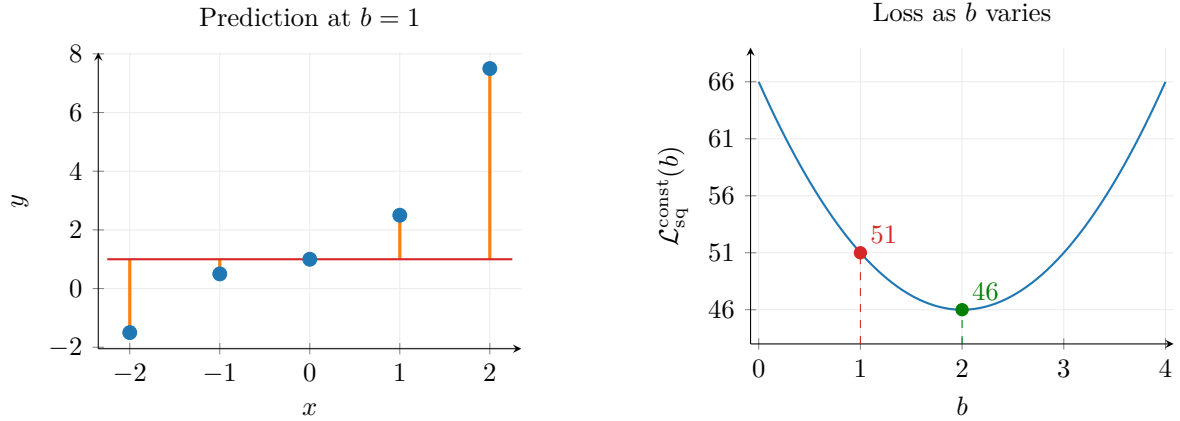


Figure 5: Left: the input axis is x , and the constant prediction $b = 1$ gives squared loss 51. Right: the horizontal axis is the parameter b ; the curve is $\mathcal{L}_{\text{sq}}^{\text{const}}(b) = 5b^2 - 20b + 66$. Its minimum is 46, attained at $b = 2$.

To differentiate (8), define

$$r_i(b) = b - y_i, \quad q(s) = s^2. \quad (9)$$

In (9), $r'_i(b) = 1$, because y_i is fixed, and $q'(s) = 2s$. Applying the chain rule to each term gives

$$(\mathcal{L}_{\text{sq}}^{\text{const}})'(b) = \sum_{i=1}^N q'(r_i(b)) r'_i(b) = 2 \sum_{i=1}^N (b - y_i) = 2 \left(Nb - \sum_{i=1}^N y_i \right). \quad (10)$$

In the final equality of (10), we used the fact that b does not depend on i .

We now set (10) to zero (in one dimension, the gradient is just the derivative) and obtain

$$Nb = \sum_{i=1}^N y_i. \quad (11)$$

Since $N \geq 1$, solving (11) gives the sample mean

$$b^* = \bar{y}, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i. \quad (12)$$

We now prove that the stationary point b^* is in fact the global minimiser. Notice that (8) is a quadratic in b with positive leading coefficient N . Hence, its graph is an upward-opening parabola, so the objective admits a global minimiser. By Fermat's theorem, any global minimiser must be a stationary point, and since b^* is the only stationary point, it follows that b^* is the global minimiser:

$$\operatorname{argmin}_{b \in \mathbb{R}} \mathcal{L}_{\text{sq}}^{\text{const}}(b) = \{b^*\} = \{\bar{y}\}.$$

Worked example. Applying (12) to our five targets gives

$$b^* = \frac{-1.5 + 0.5 + 1 + 2.5 + 7.5}{5} = 2.$$

Its residuals are 3.5, 1.5, 1, -0.5 , -5.5 , giving loss

$$3.5^2 + 1.5^2 + 1^2 + (-0.5)^2 + (-5.5)^2 = 46.$$

We remark that, for a fixed predictor family and dataset, the choice of the loss function can change the value of the best parameter. In particular, the fact that the best parameter here is the sample mean $\bar{y}$ is a consequence of our choice of the squared error as the loss function.

We will see in Exercise 5 that choosing the absolute error as our loss function instead leads to a *median* of the targets. Thus, what is considered “best” depends crucially on the choice of the loss function.

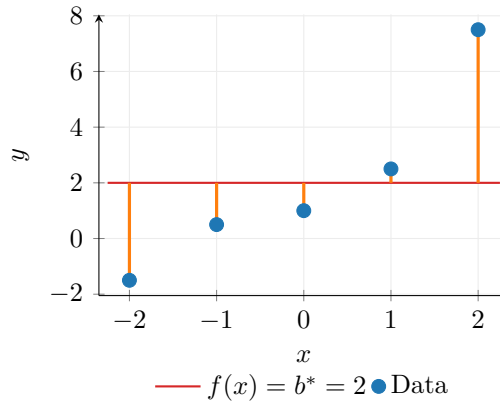


Figure 6: The best constant prediction is the mean of the targets, $b^* = (-1.5 + 0.5 + 1 + 2.5 + 7.5)/5 = 2$. It minimises the sum of squared vertical residuals, giving loss 46.

3.2 The best line through the origin

Next consider a line through the origin:

$$f_a^{\text{slope}}(x) := ax. \quad (13)$$

Using (5), its squared-error objective is the one-variable function

$$\mathcal{L}_{\text{sq}}^{\text{slope}}(a) := \sum_{i=1}^N \ell_{\text{sq}}(f_a^{\text{slope}}(x_i), y_i) = \sum_{i=1}^N (ax_i - y_i)^2. \quad (14)$$

To differentiate (14), define

$$r_i(a) = ax_i - y_i, \quad q(s) = s^2. \quad (15)$$

In (15), $r'_i(a) = x_i$, because x_i and y_i are fixed, and $q'(s) = 2s$. Applying the chain rule to each term gives

$$(\mathcal{L}_{\text{sq}}^{\text{slope}})'(a) = \sum_{i=1}^N q'(r_i(a)) r'_i(a) = 2 \sum_{i=1}^N (ax_i - y_i) x_i = 2 \left(a \sum_{i=1}^N x_i^2 - \sum_{i=1}^N x_i y_i \right). \quad (16)$$

Assume first that

$$\sum_{i=1}^N x_i^2 > 0.$$

We set (16) to zero and obtain

$$a \sum_{i=1}^N x_i^2 = \sum_{i=1}^N x_i y_i. \quad (17)$$

Solving (17) gives the stationary point

$$a^* = \frac{\sum_{i=1}^N x_i y_i}{\sum_{i=1}^N x_i^2}. \quad (18)$$

We now prove that the stationary point a^* is in fact the global minimiser. Notice that (14) is a quadratic in a with positive leading coefficient $\sum_{i=1}^N x_i^2$. Hence, its graph is an upward-opening parabola, so the objective admits a global minimiser. By Fermat's theorem, any global minimiser must be a stationary point, and since a^* is the only stationary point, it follows that a^* is the global minimiser:

$$\operatorname{argmin}_{a \in \mathbb{R}} \mathcal{L}_{\text{sq}}^{\text{slope}}(a) = \{a^*\} = \left\{ \frac{\sum_{i=1}^N x_i y_i}{\sum_{i=1}^N x_i^2} \right\}.$$

If instead

$$\sum_{i=1}^N x_i^2 = 0,$$

then $x_i = 0$ for every i . Every value of a therefore gives the same predictions, all equal to zero, and the same loss

$$\mathcal{L}_{\text{sq}}^{\text{slope}}(a) = \sum_{i=1}^N y_i^2.$$

In this case, formula (18) is unavailable and every $a \in \mathbb{R}$ is a minimiser.

Worked example. For our five observations,

$$\sum_{i=1}^5 x_i^2 = 10, \quad \sum_{i=1}^5 x_i y_i = 20, \quad a^* = 2.$$

The fitted predictions are $-4, -2, 0, 2, 4$, with residuals $-2.5, -2.5, -1, -0.5, -3.5$, giving loss 26. This is smaller than the best constant loss 46. The two families impose different restrictions, so either can give the smaller fitted loss on a different dataset.

3.3 Fitting an intercept and a slope together

We now consider a general straight line. Collect the intercept and slope in the parameter vector

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ a \end{pmatrix},$$

and define

$$f_{\boldsymbol{\theta}}^{\text{aff}}(x) := b + ax. \quad (19)$$

Using (5), its squared-error objective is

$$\mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}) := \mathcal{L}_{\text{sq}}^{\text{aff}}(b, a) := \sum_{i=1}^N \ell_{\text{sq}}(f_{\boldsymbol{\theta}}^{\text{aff}}(x_i), y_i) = \sum_{i=1}^N (b + ax_i - y_i)^2. \quad (20)$$

To differentiate (20), define

$$r_i(b, a) = b + ax_i - y_i, \quad q(s) = s^2. \quad (21)$$

Since the data are fixed,

$$\frac{\partial r_i}{\partial b} = 1, \quad \frac{\partial r_i}{\partial a} = x_i, \quad q'(s) = 2s.$$

Applying the chain rule to each term gives

$$\frac{\partial \mathcal{L}_{\text{sq}}^{\text{aff}}}{\partial b}(b, a) = 2 \sum_{i=1}^N (b + ax_i - y_i) = 2 \left(Nb + a \sum_{i=1}^N x_i - \sum_{i=1}^N y_i \right), \quad (22)$$

$$\frac{\partial \mathcal{L}_{\text{sq}}^{\text{aff}}}{\partial a}(b, a) = 2 \sum_{i=1}^N x_i (b + ax_i - y_i) = 2 \left(b \sum_{i=1}^N x_i + a \sum_{i=1}^N x_i^2 - \sum_{i=1}^N x_i y_i \right). \quad (23)$$

We now set the gradient to zero. Since the gradient has two coordinates, this means setting both partial derivatives in (22)–(23) to zero. Dividing by 2 gives the stationarity conditions

$$\begin{aligned} Nb + a \sum_{i=1}^N x_i &= \sum_{i=1}^N y_i, \\ b \sum_{i=1}^N x_i + a \sum_{i=1}^N x_i^2 &= \sum_{i=1}^N x_i y_i. \end{aligned} \quad (24)$$

Write the sample means as

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \bar{y} = \frac{1}{N} \sum_{i=1}^N y_i.$$

The first equation in (24) gives

$$b = \bar{y} - a\bar{x}. \quad (25)$$

Substituting (25) into the second stationarity condition gives

$$a \left(\sum_{i=1}^N x_i^2 - N\bar{x}^2 \right) = \sum_{i=1}^N x_i y_i - N\bar{x}\bar{y}. \quad (26)$$

Assume first that the inputs are not all equal. Then

$$\sum_{i=1}^N x_i^2 - N\bar{x}^2 = \sum_{i=1}^N (x_i - \bar{x})^2 > 0,$$

so (26) has a unique solution. Hence, the unique stationary point is

$$\boxed{\begin{aligned} a^* &= \frac{\sum_{i=1}^N x_i y_i - N\bar{x}\bar{y}}{\sum_{i=1}^N x_i^2 - N\bar{x}^2} = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}, \\ b^* &= \bar{y} - a^*\bar{x}. \end{aligned}} \quad (27)$$

Equivalently, the fitted line can be written as

$$f_{\boldsymbol{\theta}^*}^{\text{aff}}(x) = \bar{y} + a^*(x - \bar{x}), \quad \boldsymbol{\theta}^* = \begin{pmatrix} b^* \\ a^* \end{pmatrix}. \quad (28)$$

In particular, notice that the fitted line passes through $(\bar{x}, \bar{y})$.

We now prove that the stationary point $\boldsymbol{\theta}^*$ is in fact the global minimiser. Its residuals are

$$r_i^* = b^* + a^*x_i - y_i, \quad i = 1, \dots, N.$$

With this new notation, the two zero-gradient conditions in (22)–(23) say precisely that

$$\sum_{i=1}^N r_i^* = 0, \quad \sum_{i=1}^N x_i r_i^* = 0. \quad (29)$$

Now, any other parameter pair can be written as $(b^* + u, a^* + v)$, where $u, v \in \mathbb{R}$. Its residual at observation i is then

$$(b^* + u) + (a^* + v)x_i - y_i = r_i^* + u + vx_i. \quad (30)$$

Expanding the squares and using (29) gives

$$\begin{aligned} \mathcal{L}_{\text{sq}}^{\text{aff}}(b^* + u, a^* + v) &= \sum_{i=1}^N (r_i^* + u + vx_i)^2 \\ &= \underbrace{\mathcal{L}_{\text{sq}}^{\text{aff}}(b^*, a^*)}_{=0} + 2u \underbrace{\sum_{i=1}^N r_i^*}_{=0} + 2v \underbrace{\sum_{i=1}^N x_i r_i^*}_{=0} + \underbrace{\sum_{i=1}^N (u + vx_i)^2}_{\geq 0} \geq \mathcal{L}_{\text{sq}}^{\text{aff}}(b^*, a^*). \end{aligned}$$

Thus $\boldsymbol{\theta}^*$ is a global minimiser. Since the inputs are not all equal, it is the unique stationary point and hence the unique global minimiser.

If instead all inputs are equal to some value c , every prediction is $b + ac$, so the objective depends on b and a only through this quantity. From Section 3.1, the best constant prediction under squared error is $\bar{y}$. Hence,

$$\operatorname{argmin}_{(b,a) \in \mathbb{R}^2} \mathcal{L}_{\text{sq}}^{\text{aff}}(b, a) = \{(b, a) \in \mathbb{R}^2 : b + ac = \bar{y}\}.$$

The fitted predictions are therefore unique, but the slope and intercept are not.

Worked example. For our five observations, $\bar{x} = 0$ and $\bar{y} = 2$, so

$$a^* = \frac{\sum_{i=1}^5 x_i y_i}{\sum_{i=1}^5 x_i^2} = \frac{20}{10} = 2, \quad b^* = 2 - 2 \cdot 0 = 2.$$

Thus the fitted line is

$$f_{\theta^*}^{\text{aff}}(x) = 2 + 2x.$$

To become more familiar with matrix notation, notice that the stationarity conditions (24) can also be written in matrix form:

$$\begin{pmatrix} N & \sum_{i=1}^N x_i \\ \sum_{i=1}^N x_i & \sum_{i=1}^N x_i^2 \end{pmatrix} \begin{pmatrix} b \\ a \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^N y_i \\ \sum_{i=1}^N x_i y_i \end{pmatrix}. \quad (31)$$

The next section generalises (31) to several input features.

4 Regression with several input features

Now, assume that the input is a row vector $\mathbf{x} \in \mathbb{R}^d$, or more explicitly $\mathbf{x} \in \mathbb{R}^{1 \times d}$, where $d \geq 1$ is the number of features. For example, its coordinates might record several measurements of a product. The target is still a *single real number*.

Given a column of weights $\mathbf{w} \in \mathbb{R}^d$, explicitly $\mathbf{w} \in \mathbb{R}^{d \times 1}$, define

$$f_{\mathbf{w}}(\mathbf{x}) := \mathbf{x}\mathbf{w} = \sum_{j=1}^d w_j x_j. \quad (32)$$

This model has no separate intercept. Section 4.3 explains how to include one.

4.1 One input vector, one row per observation

For observations $(\mathbf{x}_i, y_i)$, $i = 1, \dots, N$, stack the input rows directly to form the *design* matrix:

$$\mathbf{X} := \begin{pmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_N \end{pmatrix} \in \mathbb{R}^{N \times d}, \quad \mathbf{y} := \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^{N \times 1}, \quad X_{ij} := (\mathbf{x}_i)_j. \quad (33)$$

With the design matrix (33), the column of predictions is

$$\begin{pmatrix} f_{\mathbf{w}}(\mathbf{x}_1) \\ \vdots \\ f_{\mathbf{w}}(\mathbf{x}_N) \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1 \mathbf{w} \\ \vdots \\ \mathbf{x}_N \mathbf{w} \end{pmatrix} = \mathbf{X}\mathbf{w}, \quad (34)$$

In (34), the single-input product $\mathbf{x}_i \mathbf{w}$ has shape $(1 \times d)(d \times 1)$ and is scalar. Its value is exactly entry i of $\mathbf{X}\mathbf{w}$, because $\mathbf{x}_i$ is row i of $\mathbf{X}$.

Define the residual vector by

$$\mathbf{r}(\mathbf{w}) := \mathbf{X}\mathbf{w} - \mathbf{y}. \quad (35)$$

Using (35), the objective becomes

$$\mathcal{L}_{\text{sq}}(\mathbf{w}) := \sum_{i=1}^N \ell_{\text{sq}}(f_{\mathbf{w}}(\mathbf{x}_i), y_i) = \sum_{i=1}^N r_i(\mathbf{w})^2 = \|\mathbf{r}(\mathbf{w})\|_2^2 = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2. \quad (36)$$

We recall from Lecture 1 that the squared norm $\|\cdot\|_2^2$ is the sum of the squared coordinates. The shapes are

$$\underbrace{\mathbf{X}}_{N \times d} \underbrace{\mathbf{w}}_{d \times 1} - \underbrace{\mathbf{y}}_{N \times 1} = \underbrace{\mathbf{r}(\mathbf{w})}_{N \times 1}; \quad \mathcal{L}_{\text{sq}}(\mathbf{w}) \in \mathbb{R}.$$

The residual vector in (35) contains one entry per observation.

4.2 The least-squares formula and its dimensions

Minimising the sum of squared residuals is called *least-squares fitting*. This is the same criterion we used for the straight line, now applied to several input features.

We first compute the gradient at an arbitrary parameter column $\mathbf{w}$, keeping the data fixed. We will then set the gradient to zero to find the stationary points and show that any stationary point is in fact a global minimiser. We will also reuse the gradient formula for gradient descent in Section 7.

For the predictor (32), the residual defined in (35) has components

$$r_i(\mathbf{w}) = \sum_{k=1}^d X_{ik} w_k - y_i.$$

Differentiating a residual. For a fixed $j \in \{1, \dots, d\}$, the data do not vary with w_j , so

$$\frac{\partial r_i}{\partial w_j}(\mathbf{w}) = \frac{\partial}{\partial w_j} \left(\sum_{k=1}^d X_{ik} w_k - y_i \right) = \sum_{k=1}^d X_{ik} \frac{\partial w_k}{\partial w_j} = \sum_{k=1}^d X_{ik} \delta_{kj} = X_{ij}. \quad (37)$$

In (37), the Kronecker delta selects the term with $k = j$. This is the same coordinate-derivative calculation as Exercise 12 of Lecture 1.

Differentiating the objective. With $q(s) = s^2$, applying the chain rule and using (37) gives

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_j}(\mathbf{w}) = \sum_{i=1}^N q'(r_i(\mathbf{w})) \frac{\partial r_i}{\partial w_j}(\mathbf{w}) = 2 \sum_{i=1}^N X_{ij} r_i(\mathbf{w}). \quad (38)$$

Equation (38) gives one scalar for each weight. To recognise the corresponding matrix–vector product, notice that, for each $j \in \{1, \dots, d\}$,

$$(\mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y}))_j = (\mathbf{X}^\top \mathbf{r}(\mathbf{w}))_j = \sum_{i=1}^N (\mathbf{X}^\top)_{ji} r_i(\mathbf{w}) = \sum_{i=1}^N X_{ij} r_i(\mathbf{w}). \quad (39)$$

Using (39) to collect the coordinate derivatives in (38) gives the column gradient

$$\boxed{\nabla \mathcal{L}_{\text{sq}}(\mathbf{w}) = 2 \mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y})}. \quad (40)$$

The shapes are

$$(d \times N)(N \times 1) = d \times 1,$$

as required for one derivative per weight.

Setting the gradient to zero. We now set the gradient in (40) to zero to find the stationary points. Equivalently, each coordinate derivative in (38) must vanish:

$$2 \sum_{i=1}^N X_{ij} r_i(\mathbf{w}) = 0, \quad j = 1, \dots, d. \quad (41)$$

By (40), collecting these conditions into one vector equation gives

$$\mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0},$$

or, equivalently, the linear system known as the *normal equations*:

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y}. \quad (42)$$

Thus, the solutions of the normal equations are precisely the stationary points of the least-squares objective. Notice that (42) represents d equations in the d weights.

Why is a stationary point a global minimiser? We now prove that any stationary point is in fact a global minimiser. Let $\mathbf{w}^*$ be a solution of (42) and write its residual column as

$$\mathbf{r}^* = \mathbf{X}\mathbf{w}^* - \mathbf{y}.$$

Since $\mathbf{w}^*$ satisfies the normal equations,

$$\mathbf{X}^\top \mathbf{r}^* = \mathbf{0}.$$

The argument proceeds as in the affine case, where we wrote any other parameter pair as $(b^* + u, a^* + v)$. Here, any other parameter column can be written as $\mathbf{w}^* + \mathbf{v}$, where $\mathbf{v} \in \mathbb{R}^d$ is a change to the parameters. Its residual column is

$$\mathbf{r}^* + \mathbf{X}\mathbf{v}.$$

Hence,

$$\mathcal{L}_{\text{sq}}(\mathbf{w}^* + \mathbf{v}) = \|\mathbf{r}^* + \mathbf{X}\mathbf{v}\|_2^2 = \mathcal{L}_{\text{sq}}(\mathbf{w}^*) + 2\mathbf{v}^\top \underbrace{\mathbf{X}^\top \mathbf{r}^*}_{=\mathbf{0}} + \underbrace{\|\mathbf{X}\mathbf{v}\|_2^2}_{\geq 0} \geq \mathcal{L}_{\text{sq}}(\mathbf{w}^*). \quad (43)$$

Thus, every stationary point of the least-squares objective is a global minimiser.

The explicit formula. If the columns of $\mathbf{X}$ are linearly independent,¹ then $\mathbf{X}^\top \mathbf{X}$ is invertible (see Appendix A). Hence, the normal equations linear system have the *unique* solution

$$\boxed{\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.} \quad (44)$$

Since $\mathbf{w}^*$ is the unique stationary point, it is also the unique global minimiser.

The dimensions in (44) explain the order of the operations:

$$\underbrace{\mathbf{X}^\top \mathbf{X}}_{d \times d}, \quad \underbrace{\mathbf{X}^\top \mathbf{y}}_{d \times 1}, \quad \underbrace{(\mathbf{X}^\top \mathbf{X})^{-1}}_{d \times d} \underbrace{\mathbf{X}^\top \mathbf{y}}_{d \times 1} = \underbrace{\mathbf{w}^*}_{d \times 1}.$$

¹We recall from Linear Algebra that the columns of $\mathbf{X}$ are *linearly independent* if

$$\mathbf{X}\mathbf{v} = \mathbf{0} \implies \mathbf{v} = \mathbf{0}.$$

Equivalently, no nonzero choice of coefficients gives the zero linear combination of the columns. In this case, we then say that $\mathbf{X}$ has *full column rank*.

In (44), the matrix to invert is $\mathbf{X}^\top \mathbf{X}$, which has size $d \times d$. The product $\mathbf{X} \mathbf{X}^\top$, instead, has size $N \times N$. These dimensions differ when $N \neq d$; even when $N = d$, the two products are not interchangeable in general.

From linear algebra, recall that full column rank requires $N \geq d$. On the other hand, we stress that this inequality alone is not sufficient: dependent feature columns make $\mathbf{X}^\top \mathbf{X}$ singular.

It can be shown that the normal equations (42) always have at least one solution. If the columns of $\mathbf{X}$ are linearly dependent, the stationary point, and hence the minimiser, need not be unique. In that case, there are infinitely many solutions of the normal equations, but they all give the same fitted predictions on the observed inputs. This is analogous to the degenerate case in Section 3.3, where all inputs were equal: different choices of the intercept and slope could give exactly the same fitted predictions because only the combination $b + ac$ mattered.

We conclude this section by noticing that with two input features, a linear predictor describes a plane in three-dimensional space, as in Figure 7. Each observation has two input coordinates and one target value.

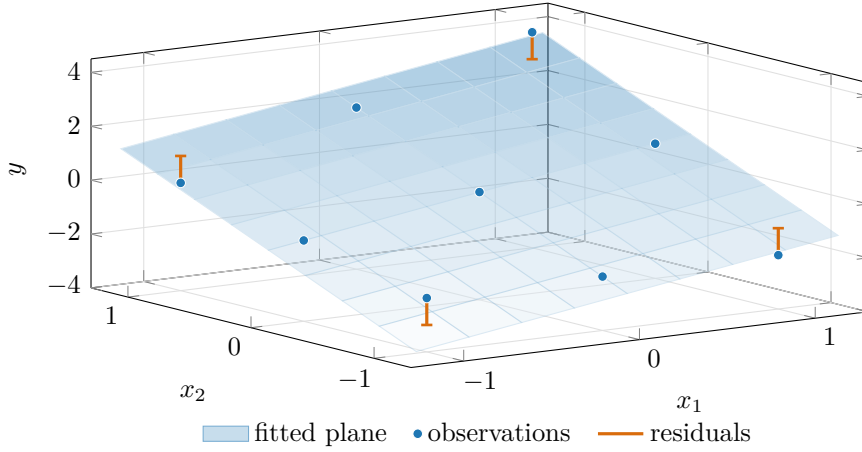


Figure 7: With two input features, a linear predictor describes a plane. For these nine observations, the least-squares fit is $f_{\mathbf{w}}(x_1, x_2) = x_1 + 2x_2$, with weights $\mathbf{w} = (1, 2)^\top$. The orange segments join each observation to its prediction on the plane, keeping both input coordinates fixed. Their lengths are the absolute residuals. Five observations lie exactly on the plane and have zero residual.

4.3 Adding an intercept with a constant feature

For the scalar-input predictor $b + ax$, use the feature vector

$$\tilde{\mathbf{x}} := \begin{pmatrix} 1 & x \end{pmatrix}, \quad \boldsymbol{\theta} := \begin{pmatrix} b \\ a \end{pmatrix}.$$

Then $\tilde{\mathbf{x}}\boldsymbol{\theta} = b + ax$. For a scalar-input dataset, the corresponding design matrix is

$$\widetilde{\mathbf{X}} := \begin{pmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_N \end{pmatrix}. \quad (45)$$

In (45), the first column is constant and the second contains the input values. Applying (42) with $\widetilde{\mathbf{X}}$ in place of $\mathbf{X}$ and $\boldsymbol{\theta}$ in place of $\mathbf{w}$ reproduces exactly (31).

Worked example. For our five observations,

$$\widetilde{\mathbf{X}} = \begin{pmatrix} 1 & -2 \\ 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} -1.5 \\ 0.5 \\ 1 \\ 2.5 \\ 7.5 \end{pmatrix},$$

and

$$\widetilde{\mathbf{X}}^\top \widetilde{\mathbf{X}} = \begin{pmatrix} 5 & 0 \\ 0 & 10 \end{pmatrix}, \quad \widetilde{\mathbf{X}}^\top \mathbf{y} = \begin{pmatrix} 10 \\ 20 \end{pmatrix}.$$

Thus the unique best-fitting straight line has

$$\begin{pmatrix} b^* \\ a^* \end{pmatrix} = \begin{pmatrix} 1/5 & 0 \\ 0 & 1/10 \end{pmatrix} \begin{pmatrix} 10 \\ 20 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.$$

The fitted line is $f_{\theta^*}^{\text{aff}}(x) = 2 + 2x$. Its predictions are $-2, 0, 2, 4, 6$, so its loss is

$$(-0.5)^2 + (-0.5)^2 + 1^2 + 1.5^2 + (-1.5)^2 = 6.$$

This improves on both restricted families and on the initial guess $f(x) = 1 + 2x$, whose loss was 11. The best-fitting line does not need to pass through any of the observed points; it minimises their total squared residual.

The same idea adds an intercept to a model with d input features. For an input row $\mathbf{x} \in \mathbb{R}^{1 \times d}$, define the augmented feature row and parameter column by

$$\tilde{\mathbf{x}} := \begin{pmatrix} 1 & \mathbf{x} \end{pmatrix} \in \mathbb{R}^{1 \times (d+1)}, \quad \boldsymbol{\theta} := \begin{pmatrix} b \\ \mathbf{w} \end{pmatrix} \in \mathbb{R}^{(d+1) \times 1}. \quad (46)$$

Then

$$\tilde{\mathbf{x}}\boldsymbol{\theta} = b + \mathbf{x}\mathbf{w}. \quad (47)$$

For N observations, this corresponds to augmenting the design matrix with a first column of ones. Let

$$\mathbf{1}_N := \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix} \in \mathbb{R}^{N \times 1}.$$

Then the augmented design matrix is

$$\widetilde{\mathbf{X}} := \begin{pmatrix} 1 & \mathbf{x}_1 \\ \vdots & \vdots \\ 1 & \mathbf{x}_N \end{pmatrix} = \begin{pmatrix} \mathbf{1}_N & \mathbf{X} \end{pmatrix} \in \mathbb{R}^{N \times (d+1)}. \quad (48)$$

Hence

$$\widetilde{\mathbf{X}}\boldsymbol{\theta} = b\mathbf{1}_N + \mathbf{X}\mathbf{w},$$

whose i -th entry is $b + \mathbf{x}_i\mathbf{w}$.

Thus, adding an intercept is equivalent to adding a constant feature equal to 1. The original design matrix $\mathbf{X}$ contains only the d input features, whereas the augmented design matrix $\widetilde{\mathbf{X}}$ contains these d features together with the additional constant feature.

4.4 Regression with nonlinear features

The idea of the previous section extends much further. In the affine case, we reinterpreted the intercept as the coefficient of an additional constant feature 1, so that the predictor $b + ax$ became a linear combination of the two features 1 and x . More generally, we can replace the original input by a collection of fixed features constructed from it, and then fit a linear combination of those features.

For example, the predictor

$$f_{\mathbf{w}}^{\text{quad}}(x) := w_1 + w_2x + w_3x^2 \quad (49)$$

is obtained by using the three features 1, x , and x^2 . Although this predictor is quadratic as a function of the original input x when $w_3 \neq 0$, it is linear in the adjustable coefficients w_1, w_2, w_3 .

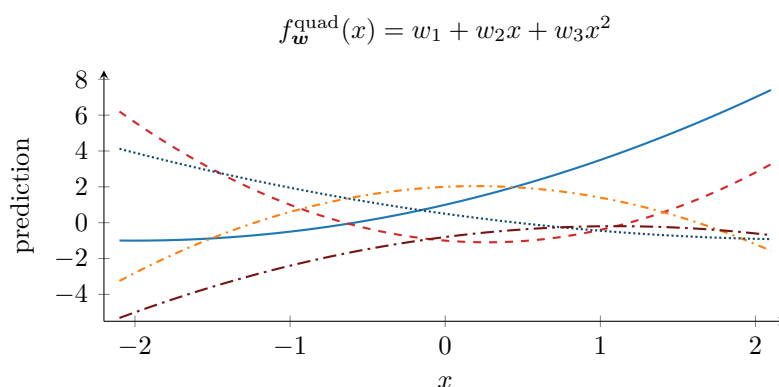


Figure 8: Different choices of w_1, w_2, w_3 give different quadratic predictors. These curves show possible members of the family, before fitting the coefficients to any data.

This motivates the following general construction. Choose fixed functions $\phi_1, \dots, \phi_m : \mathbb{R} \rightarrow \mathbb{R}$, called *features*, and collect them in the *feature map*

$$\phi : \mathbb{R} \longrightarrow \mathbb{R}^m, \quad \phi(x) = \begin{pmatrix} \phi_1(x) & \cdots & \phi_m(x) \end{pmatrix}. \quad (50)$$

Thus, instead of using the original input x directly, we represent it through the m quantities $\phi_1(x), \dots, \phi_m(x)$.

With a column of coefficients $\mathbf{w} \in \mathbb{R}^m$, we then define

$$f_{\mathbf{w}}^{\phi}(x) := \phi(x)\mathbf{w} = \sum_{j=1}^m w_j \phi_j(x). \quad (51)$$

The predictor is therefore linear in the adjustable coefficients $w_1, \dots, w_m$, even though the features themselves may be nonlinear functions of the original input.

If one of the features is the constant function 1, its coefficient plays the role of an intercept. Thus, no additional intercept needs to be introduced separately. For example, the quadratic predictor (49) corresponds to

$$\phi(x) = \begin{pmatrix} 1 & x & x^2 \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \end{pmatrix}.$$

For the observed inputs $x_1, \dots, x_N$, place the transformed feature rows into the *transformed design matrix*

$$\mathbf{\Phi} := \begin{pmatrix} \phi(x_1) \\ \vdots \\ \phi(x_N) \end{pmatrix} = \begin{pmatrix} \phi_1(x_1) & \cdots & \phi_m(x_1) \\ \vdots & & \vdots \\ \phi_1(x_N) & \cdots & \phi_m(x_N) \end{pmatrix} \in \mathbb{R}^{N \times m}. \quad (52)$$

Its entries are

$$\Phi_{ij} = \phi_j(x_i).$$

The vector of predictions on the observed inputs is therefore

$$\mathbf{\Phi} \mathbf{w} \in \mathbb{R}^{N \times 1},$$

and the corresponding empirical squared loss is

$$\mathcal{L}_{\text{sq}}^\phi(\mathbf{w}) := \sum_{i=1}^N \ell_{\text{sq}}(f_{\mathbf{w}}^\phi(x_i), y_i) = \|\mathbf{\Phi} \mathbf{w} - \mathbf{y}\|_2^2. \quad (53)$$

At this point, the fitting problem has exactly the same form as the linear least-squares problem of Section 4.2, with $\mathbf{\Phi}$ in place of $\mathbf{X}$. Hence its stationary points satisfy the normal equations

$$\mathbf{\Phi}^\top \mathbf{\Phi} \mathbf{w} = \mathbf{\Phi}^\top \mathbf{y}. \quad (54)$$

As we proved in Section 4.2, every solution of these equations is a global minimiser. If $\mathbf{\Phi}$ has full column rank, the minimiser is unique and is given by

$$\mathbf{w}^* = (\mathbf{\Phi}^\top \mathbf{\Phi})^{-1} \mathbf{\Phi}^\top \mathbf{y}. \quad (55)$$

For the quadratic predictor (49), the feature map and transformed design matrix are

$$\phi(x) = \begin{pmatrix} 1 & x & x^2 \end{pmatrix}, \quad \mathbf{\Phi} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ \vdots & \vdots & \vdots \\ 1 & x_N & x_N^2 \end{pmatrix}. \quad (56)$$

The word *linear* in this fitting problem refers to the adjustable coefficients, not necessarily to the dependence on the original input. For example, $w_1 + w_2x + w_3x^2$ is nonlinear in x when $w_3 \neq 0$, but it is linear in the adjustable coefficients (w_1, w_2, w_3) . By contrast, a model such as $\sin(wx)$ is not of the fixed-feature linear form $w\phi(x)$.²

²Indeed, if there were a function ϕ , independent of w , such that $\sin(wx) = w\phi(x)$ for all w, x , then setting $w = 1$ would give $\phi(x) = \sin x$. Hence we would have $\sin(wx) = w \sin x$ for all w, x , which is false; for example, with $x = \pi/2$ and $w = 2$, the two sides are 0 and 2.

Worked example. Using the features (56) for our five observations, the transformed matrix and its normal equations (54) are

$$\Phi = \begin{pmatrix} 1 & -2 & 4 \\ 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad \underbrace{\begin{pmatrix} 5 & 0 & 10 \\ 0 & 10 & 0 \\ 10 & 0 & 34 \end{pmatrix}}_{\Phi^T \Phi} \mathbf{w} = \underbrace{\begin{pmatrix} 10 \\ 20 \\ 27 \end{pmatrix}}_{\Phi^T \mathbf{y}}.$$

The columns of Φ are independent, so the solution is unique. We can find it by *Gaussian elimination*, without computing the inverse of $\Phi^T \Phi$. In (57), the last column contains the right-hand side. Subtracting twice the first row from the third row gives

$$\left[\begin{array}{ccc|c} 5 & 0 & 10 & 10 \\ 0 & 10 & 0 & 20 \\ 10 & 0 & 34 & 27 \end{array} \right] \xrightarrow{R_3 \leftarrow R_3 - 2R_1} \left[\begin{array}{ccc|c} 5 & 0 & 10 & 10 \\ 0 & 10 & 0 & 20 \\ 0 & 0 & 14 & 7 \end{array} \right]. \quad (57)$$

The coefficient matrix on the right of (57) is upper triangular. Reading its equations from the bottom up, a procedure called *back substitution*, gives

$$w_3 = \frac{7}{14} = \frac{1}{2}, \quad w_2 = \frac{20}{10} = 2, \quad w_1 = \frac{10 - 10w_3}{5} = 1. \quad (58)$$

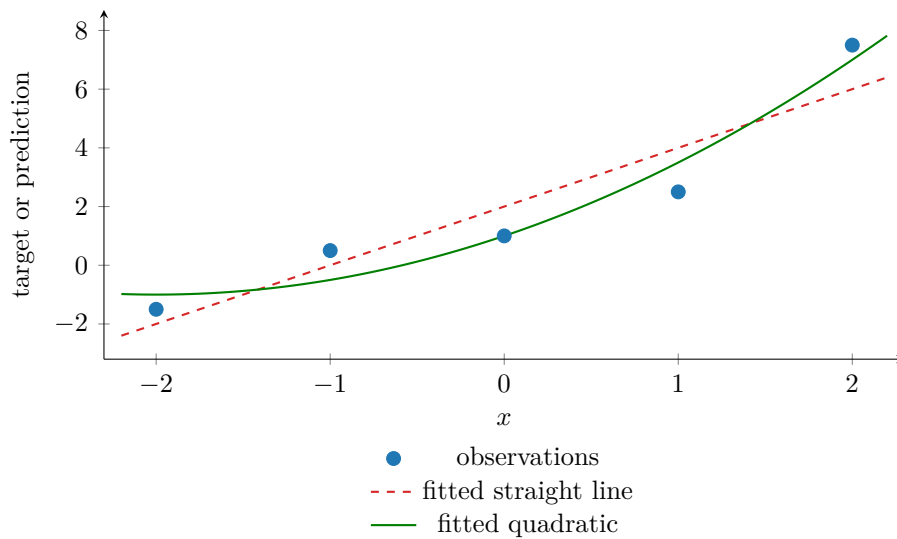
Substituting (58) into (49) gives

$$\mathbf{w}^* = \begin{pmatrix} 1 \\ 2 \\ 1/2 \end{pmatrix}, \quad f_{\mathbf{w}^*}^{\text{quad}}(x) = 1 + 2x + \frac{1}{2}x^2.$$

Its predictions are $-1, -0.5, 1, 3.5, 7$, with residuals $0.5, -1, 0, 1, -0.5$. The squared loss is

$$0.5^2 + (-1)^2 + 0^2 + 1^2 + (-0.5)^2 = \frac{5}{2},$$

which is smaller than the fitted straight-line loss 6.



5 Gradient descent

We have seen that the squared-loss minimiser for linear regression can be computed by solving the normal equations. For a dense design matrix with N observations and d linearly independent feature columns, forming the system and solving it by Gaussian elimination costs $O(Nd^2 + d^3)$ arithmetic operations.³ This can become expensive when d is large.

In Lecture 1, we introduced gradient descent as an iterative method for minimising a differentiable objective. We now apply it to the squared-error objective for linear regression and compare its updates with the explicit least-squares solution.

Gradient descent starts from an initial parameter vector and uses derivatives to guide successive updates that aim to reduce the loss. After each update we have a candidate predictor, so we can stop when the approximation is adequate for our purposes or when we have used the available computational budget.

For this regression problem, each iteration costs $O(Nd)$. An adequate approximation reached in relatively few iterations may therefore be cheaper than a direct solve. We introduce the method here with a known minimiser to check against; in later lectures, we will use it for models where a direct solution method is unavailable.

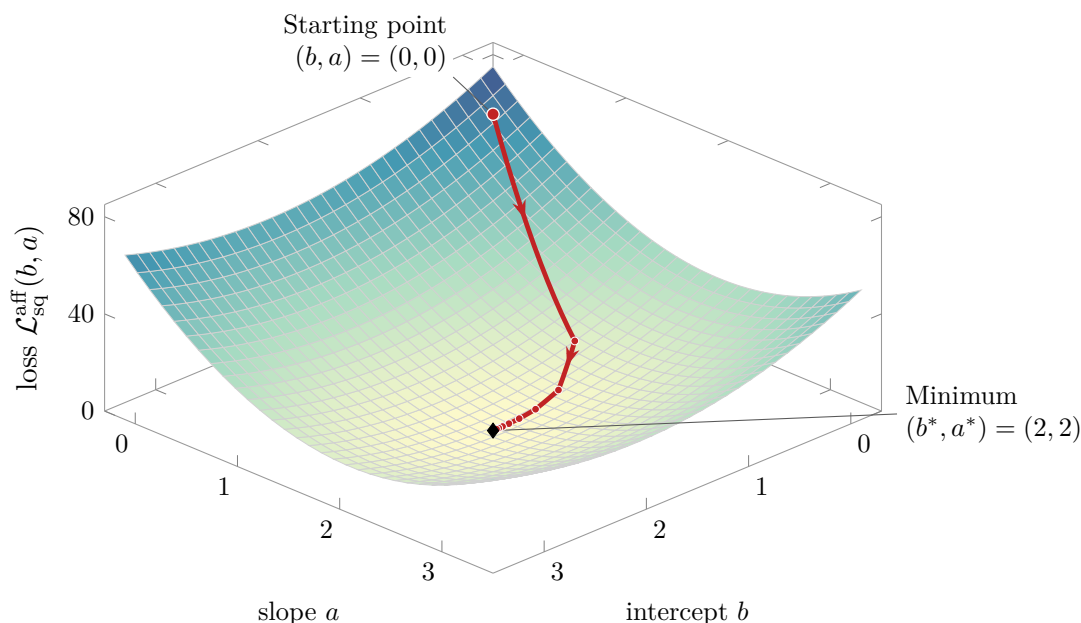


Figure 9: Gradient descent moves downhill on the loss surface. Height represents the squared-error loss for five observations. The red points are successive iterates, starting from $(b, a) = (0, 0)$ with learning rate $\eta = 0.04$; the arrows indicate their order. They approach the minimum at $(2, 2)$, where the loss is 6. Eight updates are shown.

³Here, $O(\cdot)$ indicates an upper bound on how the number of arithmetic operations grows with the problem dimensions, up to a constant factor.

Let $\mathcal{L} : \mathbb{R}^p \rightarrow \mathbb{R}$ be a differentiable objective of a column parameter $\boldsymbol{\theta}$. As in Lecture 1, its gradient has the same shape as that parameter: it is the column of its coordinate derivatives. Starting from an initial parameter vector $\boldsymbol{\theta}^{(0)}$, gradient descent updates

$$\boxed{\boldsymbol{\theta}^{(t+1)} := \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)})}, \quad t = 0, 1, 2, \dots, \quad (59)$$

where $\eta > 0$ is the *learning rate*, or step size. The superscript (t) is an iteration index, not a power and not an observation index.

5.1 Why move against the gradient?

For a small change $\boldsymbol{\Delta}$, differentiability gives the first-order approximation

$$\mathcal{L}(\boldsymbol{\theta} + \boldsymbol{\Delta}) \approx \mathcal{L}(\boldsymbol{\theta}) + \nabla \mathcal{L}(\boldsymbol{\theta})^\top \boldsymbol{\Delta}. \quad (60)$$

The last term in (60) is the sum of each coordinate change multiplied by the corresponding partial derivative. With $\boldsymbol{\Delta} = -\eta \nabla \mathcal{L}(\boldsymbol{\theta})$, it is

$$\nabla \mathcal{L}(\boldsymbol{\theta})^\top \boldsymbol{\Delta} = -\eta \|\nabla \mathcal{L}(\boldsymbol{\theta})\|_2^2.$$

If the gradient is nonzero, this is negative, and therefore a sufficiently small step in that direction reduces the objective. The phrase *sufficiently small* matters: a first-order approximation is only local and does not promise anything about arbitrarily large steps.

The step size matters. An excessively large step can increase the loss. For example, take

$$F(a) = 100a^2, \quad F'(a) = 200a. \quad (61)$$

Applying gradient descent to (61) with $\eta = 0.1$ gives

$$a^{(t+1)} = a^{(t)} - 0.1 \cdot 200a^{(t)} = -19a^{(t)}. \quad (62)$$

Under (62), the magnitude grows from any nonzero starting value. Even with a suitable step size, approaching a minimiser need not mean reaching it in finitely many steps.

6 Gradient descent for a straight line

We now apply the gradient-descent rule (59) to the affine predictor

$$f_{\boldsymbol{\theta}}^{\text{aff}}(x) = b + ax, \quad \boldsymbol{\theta} = \begin{pmatrix} b \\ a \end{pmatrix}.$$

The objective to minimise is the empirical squared loss $\mathcal{L}_{\text{sq}}^{\text{aff}}$ defined in (20). Thus, in the general update (59), we replace $\mathcal{L}$ by $\mathcal{L}_{\text{sq}}^{\text{aff}}$:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}^{(t)}). \quad (63)$$

To perform this update, we need to calculate the gradient at the current parameter column

$$\boldsymbol{\theta}^{(t)} = \begin{pmatrix} b^{(t)} \\ a^{(t)} \end{pmatrix}.$$

Since the first coordinate is the intercept b and the second is the slope a , the gradient contains the partial derivative with respect to b first, followed by the partial derivative with respect to a .

We have already calculated these derivatives in (22) and (23). Previously, we set them equal to zero to find the stationary point. This time, we evaluate them at the current values $b^{(t)}, a^{(t)}$ to determine the update.

For each observation, first calculate the residual of the current line:

$$r_i^{(t)} := b^{(t)} + a^{(t)}x_i - y_i, \quad i = 1, \dots, N. \quad (64)$$

Evaluating (22) and (23) at $(b^{(t)}, a^{(t)})$, and then using (64), gives

$$\nabla \mathcal{L}_{\text{sq}}^{\text{aff}}(\theta^{(t)}) = \begin{pmatrix} \frac{\partial \mathcal{L}_{\text{sq}}^{\text{aff}}}{\partial b}(b^{(t)}, a^{(t)}) \\ \frac{\partial \mathcal{L}_{\text{sq}}^{\text{aff}}}{\partial a}(b^{(t)}, a^{(t)}) \end{pmatrix} = 2 \begin{pmatrix} \sum_{i=1}^N (b^{(t)} + a^{(t)}x_i - y_i) \\ \sum_{i=1}^N x_i (b^{(t)} + a^{(t)}x_i - y_i) \end{pmatrix} = 2 \begin{pmatrix} \sum_{i=1}^N r_i^{(t)} \\ \sum_{i=1}^N x_i r_i^{(t)} \end{pmatrix}. \quad (65)$$

We can now substitute (65) into (63). The resulting update is

$$\begin{pmatrix} b^{(t+1)} \\ a^{(t+1)} \end{pmatrix} = \begin{pmatrix} b^{(t)} \\ a^{(t)} \end{pmatrix} - 2\eta \begin{pmatrix} \sum_{i=1}^N r_i^{(t)} \\ \sum_{i=1}^N x_i r_i^{(t)} \end{pmatrix}. \quad (66)$$

Hence, at each iteration, first compute the residuals in (64) and then apply (66) one coordinate at a time:

$$\begin{aligned} r_i^{(t)} &:= b^{(t)} + a^{(t)}x_i - y_i, \quad i = 1, \dots, N, \\ b^{(t+1)} &:= b^{(t)} - 2\eta \sum_{i=1}^N r_i^{(t)}, \end{aligned} \quad (67)$$

$$a^{(t+1)} := a^{(t)} - 2\eta \sum_{i=1}^N x_i r_i^{(t)}. \quad (68)$$

Both parameter updates use the same residuals, computed from $b^{(t)}$ and $a^{(t)}$ before either parameter is updated.

Interpreting the signs. Recall that $r_i^{(t)} < 0$ means that the current prediction at x_i is below the target, while $r_i^{(t)} > 0$ means that it is above the target.

For the intercept, if

$$\sum_i r_i^{(t)} < 0,$$

the predictions are, overall, too low. Then (67) increases b , shifting the whole line upwards.

For the slope, each observation contributes $x_i r_i^{(t)}$. If $x_i > 0$ and the prediction is too low, then $x_i r_i^{(t)} < 0$, so (68) increases the slope. If $x_i < 0$ and the prediction is too low, then $x_i r_i^{(t)} > 0$, so the update decreases the slope.

Worked example: one simultaneous update. Use our five observations, start from

$$\boldsymbol{\theta}^{(0)} = \begin{pmatrix} b^{(0)} \\ a^{(0)} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

and take $\eta = 0.04$, as in Figure 9. The initial line predicts zero at every input. Therefore, by (64), its residuals are

$$r_1^{(0)} = 1.5, \quad r_2^{(0)} = -0.5, \quad r_3^{(0)} = -1, \quad r_4^{(0)} = -2.5, \quad r_5^{(0)} = -7.5.$$

The two sums needed in (65) are

$$\sum_{i=1}^5 r_i^{(0)} = -10, \quad \sum_{i=1}^5 x_i r_i^{(0)} = -20.$$

Hence,

$$\nabla \mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}^{(0)}) = 2 \begin{pmatrix} -10 \\ -20 \end{pmatrix} = \begin{pmatrix} -20 \\ -40 \end{pmatrix}. \quad (69)$$

Substituting (69) into (63) gives

$$\boldsymbol{\theta}^{(1)} = \begin{pmatrix} b^{(1)} \\ a^{(1)} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - 0.04 \begin{pmatrix} -20 \\ -40 \end{pmatrix} = \begin{pmatrix} 0.8 \\ 1.6 \end{pmatrix}. \quad (70)$$

The new line is therefore

$$f_{\boldsymbol{\theta}^{(1)}}^{\text{aff}}(x) = 0.8 + 1.6x.$$

Its predictions are $-2.4, -0.8, 0.8, 2.4, 4$, with residuals $-0.9, -1.3, -0.2, -0.1, -3.5$. The empirical loss decreases from 66 to

$$(-0.9)^2 + (-1.3)^2 + (-0.2)^2 + (-0.1)^2 + (-3.5)^2 = 14.8.$$

The parameters are not optimal after one step: the analytic solution was $(b^*, a^*) = (2, 2)$, with empirical loss 6. To perform the next iteration, we repeat the same calculation using the residuals of the new line.

7 Gradient descent with several features

We now apply the same gradient-descent rule to the linear predictor

$$f_{\mathbf{w}}(\mathbf{x}) = \mathbf{x}\mathbf{w}$$

with d feature weights. The objective is the empirical squared loss from (36):

$$\mathcal{L}_{\text{sq}}(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2.$$

Here the parameter column is $\mathbf{w}$, so we use $\mathbf{w}$ in place of $\boldsymbol{\theta}$ and $\mathcal{L}_{\text{sq}}$ in place of $\mathcal{L}$ in (59). This gives

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \mathcal{L}_{\text{sq}}(\mathbf{w}^{(t)}). \quad (71)$$

To perform this update, we need the gradient evaluated at the current weights $\mathbf{w}^{(t)}$. We already derived the required formula in (40):

$$\nabla \mathcal{L}_{\text{sq}}(\mathbf{w}) = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}).$$

This formula is valid at every parameter column $\mathbf{w}$. Previously, we set it equal to zero to find the stationary points; now we evaluate it at the current weights to determine the gradient-descent step.

At iteration t , the current predictions are

$$\mathbf{X}\mathbf{w}^{(t)}.$$

At observation i , this prediction is

$$f_{\mathbf{w}^{(t)}}(\mathbf{x}_i) = \mathbf{x}_i\mathbf{w}^{(t)} = \sum_{k=1}^d X_{ik}w_k^{(t)}.$$

Subtracting the targets gives the current residual column

$$\mathbf{r}^{(t)} := \mathbf{X}\mathbf{w}^{(t)} - \mathbf{y},$$

whose i -th entry is

$$r_i^{(t)} = \sum_{k=1}^d X_{ik}w_k^{(t)} - y_i.$$

Evaluating (40) at the current weights therefore gives

$$\nabla \mathcal{L}_{\text{sq}}(\mathbf{w}^{(t)}) = 2\mathbf{X}^\top(\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y}) = 2\mathbf{X}^\top\mathbf{r}^{(t)}. \quad (72)$$

Equivalently, the coordinate formula (38) gives

$$\frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_j}(\mathbf{w}^{(t)}) = 2 \sum_{i=1}^N X_{ij}r_i^{(t)}, \quad j = 1, \dots, d.$$

Thus, the derivative with respect to weight w_j is twice the sum of the current residuals multiplied by the corresponding feature values X_{ij} .

Substituting (72) into (71), each iteration can be written as

$$\mathbf{r}^{(t)} := \mathbf{X}\mathbf{w}^{(t)} - \mathbf{y}, \quad (73)$$

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - 2\eta\mathbf{X}^\top\mathbf{r}^{(t)}. \quad (74)$$

Equivalently, in coordinates,

$$\begin{aligned} r_i^{(t)} &:= \sum_{k=1}^d X_{ik}w_k^{(t)} - y_i, & i = 1, \dots, N, \\ w_j^{(t+1)} &:= w_j^{(t)} - 2\eta \sum_{i=1}^N X_{ij}r_i^{(t)}, & j = 1, \dots, d. \end{aligned} \quad (75)$$

Thus, at each iteration, we first compute all residuals from the current weight column $\mathbf{w}^{(t)}$, and then use those same residuals to update all d weights simultaneously. Only after all weights have been updated do we recompute the predictions and residuals for the next iteration. The data $\mathbf{X}, \mathbf{y}$ remain fixed throughout.

Computing one iteration requires two matrix-vector products: $\mathbf{X}\mathbf{w}^{(t)}$ for the predictions and $\mathbf{X}^\top\mathbf{r}^{(t)}$ for the gradient. For a dense design matrix, each costs $O(Nd)$ arithmetic operations. Subtracting the targets costs $O(N)$, and updating the weights costs $O(d)$, so the total cost per iteration is $O(Nd)$.

Including an intercept. Finally, consider the model with an explicit intercept from (47):

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = b + \mathbf{x}\mathbf{w}.$$

Using the augmented representation introduced in (46), the complete parameter column and augmented design matrix are

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ \mathbf{w} \end{pmatrix}, \quad \widetilde{\mathbf{X}} = \begin{pmatrix} \mathbf{1}_N & \mathbf{X} \end{pmatrix},$$

where $\mathbf{1}_N$ is the column vector of N ones. The empirical squared loss is

$$\mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}) = \left\| \widetilde{\mathbf{X}}\boldsymbol{\theta} - \mathbf{y} \right\|_2^2.$$

We therefore apply the same gradient formula (40), now with $\widetilde{\mathbf{X}}$ in place of $\mathbf{X}$ and $\boldsymbol{\theta}$ in place of $\mathbf{w}$:

$$\nabla \mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}) = 2\widetilde{\mathbf{X}}^{\top}(\widetilde{\mathbf{X}}\boldsymbol{\theta} - \mathbf{y}). \quad (76)$$

At iteration t , first compute the residual column

$$\mathbf{r}^{(t)} := \widetilde{\mathbf{X}}\boldsymbol{\theta}^{(t)} - \mathbf{y} = b^{(t)}\mathbf{1}_N + \mathbf{X}\mathbf{w}^{(t)} - \mathbf{y}. \quad (77)$$

Since the first column of $\widetilde{\mathbf{X}}$ consists of ones, evaluating (76) gives

$$\nabla \mathcal{L}_{\text{sq}}^{\text{aff}}(\boldsymbol{\theta}^{(t)}) = 2 \begin{pmatrix} \mathbf{1}_N^{\top} \mathbf{r}^{(t)} \\ \mathbf{X}^{\top} \mathbf{r}^{(t)} \end{pmatrix} = 2 \begin{pmatrix} \sum_{i=1}^N r_i^{(t)} \\ \mathbf{X}^{\top} \mathbf{r}^{(t)} \end{pmatrix}.$$

Thus, the complete iteration is

$$\begin{aligned} \mathbf{r}^{(t)} &:= b^{(t)}\mathbf{1}_N + \mathbf{X}\mathbf{w}^{(t)} - \mathbf{y}, \\ b^{(t+1)} &:= b^{(t)} - 2\eta \sum_{i=1}^N r_i^{(t)}, \\ \mathbf{w}^{(t+1)} &:= \mathbf{w}^{(t)} - 2\eta \mathbf{X}^{\top} \mathbf{r}^{(t)}. \end{aligned} \quad (78)$$

Nonlinear features. We conclude by briefly explaining how the same gradient-descent calculation extends to the fixed nonlinear features introduced in Section 4.4. Recall that their empirical squared loss is

$$\mathcal{L}_{\text{sq}}^{\phi}(\mathbf{w}) = \left\| \Phi \mathbf{w} - \mathbf{y} \right\|_2^2,$$

so this has exactly the same form as the linear-regression objective, with Φ in place of $\mathbf{X}$.

Therefore, at iteration t , we first compute the residuals

$$\mathbf{r}^{(t)} := \Phi \mathbf{w}^{(t)} - \mathbf{y}, \quad (79)$$

and then update the feature weights according to

$$\mathbf{w}^{(t+1)} := \mathbf{w}^{(t)} - 2\eta \Phi^{\top} \mathbf{r}^{(t)}. \quad (80)$$

Thus, once the nonlinear features have been fixed, gradient descent proceeds exactly as before: the only change is that the original design matrix $\mathbf{X}$ is replaced by the transformed design matrix Φ .

8 Exercises

The first three exercises use the dataset below. The exercises can be done by hand with a calculator.

i	1	2	3	4	5
x_i	-2	-1	0	1	2
y_i	-6.2	-2.6	0.5	2.7	5.7

- 1. Compare three predictors.** Compute the predictions and sum-of-squared-errors value for each function

$$f_1(x) = x, \quad f_2(x) = 2x, \quad f_3(x) = 3x.$$

Which gives the smallest loss on the displayed dataset? Does this establish that it is the best predictor among all straight lines?

- 2. Fit a straight line.** Fit the family $f_{\theta}^{\text{aff}}(x) = b + ax$ by squared-error minimisation. Use the feature row $(1, x)$ and the matrix method in Section 4. Construct the design matrix, compute its product with its transpose in the correct order, compute its transpose times $\mathbf{y}$, and solve for (b, a) . Check that the minimiser is unique.

- 3. Fit a quadratic without an intercept.** Fit

$$f_w(x) = w_1x + w_2x^2$$

by the same squared-error criterion. Specify the two feature functions, form Φ , and compute the fitted coefficients. This family has no constant term: do not add a column of ones. Check the condition needed for the inverse formula (55).

- 4. Differentiate the absolute value.** Let $q : \mathbb{R} \rightarrow \mathbb{R}$ be $q(s) = |s|$. Sketch its graph and find $q'(s)$ separately for $s < 0$ and $s > 0$. Explain why the derivative does not exist at zero. Express the derivative at nonzero inputs using

$$\text{sign}(s) = \begin{cases} -1, & s < 0, \\ 0, & s = 0, \\ 1, & s > 0. \end{cases} \quad (81)$$

The convention $\text{sign}(0) = 0$ in (81) is not a statement that $q'(0)$ exists.

5. Changing the loss for the constant predictor. Return to the five observations used in the worked examples:

i	1	2	3	4	5
x_i	-2	-1	0	1	2
y_i	-1.5	0.5	1	2.5	7.5

In Section 3.1, we fitted the constant predictor from (7) using the squared loss ℓ_{sq} from (4). We found that the best constant is the mean $\bar{y} = 2$, with $\mathcal{L}_{\text{sq}}^{\text{const}}(2) = 46$.

What changes if we use the absolute loss ℓ_{abs} from (4) instead? Keeping the same data and family of predictors, we now minimise

$$\mathcal{L}_{\text{abs}}^{\text{const}}(b) = \sum_{i=1}^5 \ell_{\text{abs}}(f_b^{\text{const}}(x_i), y_i) = \sum_{i=1}^5 |b - y_i|. \quad (82)$$

- For b different from all five targets, express the derivative of (82) using the sign function (81) from Exercise 4. Interpret it as the number of targets below b minus the number above it. Find the slope on each interval between successive targets, and on the two outer intervals. Sketch $\mathcal{L}_{\text{abs}}^{\text{const}}$, find its minimiser b_{abs}^* , and relate it to the median of the five targets. Does $(\mathcal{L}_{\text{abs}}^{\text{const}})'(b_{\text{abs}}^*)$ exist?
- Compare your fitted constant with the squared-loss fit already obtained in Section 3.1. Complete the table; the known squared-error value is supplied:

constant	$\mathcal{L}_{\text{abs}}^{\text{const}}(b)$	$\mathcal{L}_{\text{sq}}^{\text{const}}(b)$
b_{abs}^*		
$\bar{y} = 2$		46

Which constant is preferred under each loss? Compare entries within each column; the two columns measure error differently.

- Plot the five observations and the two horizontal prediction lines $y = b_{\text{abs}}^*$ and $y = \bar{y}$ on the same axes. Explain how changing the loss has changed the optimal predictor, referring to the way the two losses penalise large errors in Section 2.

Reflect on what this comparison shows: choosing a loss determines what we optimise for and can change which predictor is best. If the loss does not reflect the goal of the application, even a predictor that minimises it may be a poor choice for that goal.

Hint for (a). As b crosses one target from left to right, that term's slope changes from -1 to $+1$. At a corner, inspect the slopes on either side; do not assume that a minimiser must have a derivative.

A Invertibility of the normal-equation matrix

Let $\mathbf{X} \in \mathbb{R}^{N \times d}$ have linearly independent columns. We show that the square matrix $\mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{d \times d}$ is invertible, as used in Section 4.2.

We use the following standard linear-algebra result: a square matrix $\mathbf{G}$ is invertible if and only if the only solution of $\mathbf{G}\mathbf{v} = \mathbf{0}$ is $\mathbf{v} = \mathbf{0}$. Thus, take a column vector $\mathbf{v} \in \mathbb{R}^d$ satisfying

$$\mathbf{X}^\top \mathbf{X} \mathbf{v} = \mathbf{0}. \quad (83)$$

Multiplying (83) on the left by $\mathbf{v}^\top$ gives

$$0 = \mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = (\mathbf{X} \mathbf{v})^\top (\mathbf{X} \mathbf{v}) = \|\mathbf{X} \mathbf{v}\|_2^2. \quad (84)$$

The squared norm in (84) is a sum of squared coordinates. It can be zero only when every coordinate is zero, so

$$\mathbf{X} \mathbf{v} = \mathbf{0}. \quad (85)$$

This is where we use the independence of the columns. Writing them as $\mathbf{c}_1, \dots, \mathbf{c}_d \in \mathbb{R}^N$, equation (85) means

$$v_1 \mathbf{c}_1 + \dots + v_d \mathbf{c}_d = \mathbf{0}. \quad (86)$$

By linear independence, all coefficients in (86) must be zero: $v_1 = \dots = v_d = 0$. Hence $\mathbf{v} = \mathbf{0}$. We have therefore proved

$$\mathbf{X}^\top \mathbf{X} \mathbf{v} = \mathbf{0} \implies \mathbf{v} = \mathbf{0},$$

which establishes that $\mathbf{X}^\top \mathbf{X}$ is invertible.